

Assignment Kit for Coding Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material is approved for public release.
Distribution limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements

Produce, document, and submit a completed coding standard that calls for quality coding practices.

For LOC counting, ensure that a separate physical source line is used for each logical line of code.

Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example

- Since it is an example, tailor it to meet your personal needs.
 - If you have an existing organizational standard, consider using it for the PSP exercises.
-

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> / *****/ /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ / *****/ </pre>
Listing Contents	Provide a summary of the listing contents
Contents Example	<pre> / *****/ /* Listing Contents: */ /* Reuse instructions */ /* Modification instructions */ /* Compilation instructions */ /* Includes */ /* Class declarations: */ /* CData */ /* ASet */ /* Source code in c:/classes/CData.cpp: */ /* CData */ /* CData() */ /* Empty() */ / *****/ </pre>

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	<pre> / ***** */ /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ / ***** */ </pre>
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	<pre> Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */ </pre>
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	<pre> If(record_count > limit) /* have all records been processed? */ </pre>
Bad Comment	<pre> If(record_count > limit) /* check if record count exceeds limit */ </pre>
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	<pre> / ***** */ /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ / ***** */ </pre>
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.
Indenting Example	<pre> while (miss_distance > threshold) { success_code = move_robot (target_location); if (success_code == MOVE_FAILED) { printf("The robot move has failed.\n"); } } </pre>
Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization	<pre>#define DEFAULT-NUMBER-OF-STUDENTS 15</pre>

Examples	<code>int class-size = DEFAULT-NUMBER-OF-STUDENTS;</code>
-----------------	---

Evaluation criteria and suggestions

**Evaluation
criteria**

Your standard must be

- complete
 - legible
-

Suggestions

Keep your standards simple and short.

Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	To guide the development of programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> * * @version 2.0.1 26/11/2025 * @author Bridget Mendez </pre>
Listing Contents	Provide a summary of the listing contents.
Contents Example	<pre> /** * Ejecuta el Test 1: regresión entre tamaño estimado y * tamaño real. */ public void ejecutarTest1() { Data dataX = new Data(); Data dataY = new Data(); Input input = new Input(); input.readData("estimated.txt", dataX); input.readData("actualAdded.txt", dataY); procesarDatos(dataX, dataY, XK_VALUE, "Tamaño Estimado", "Tamaño Real"); } </pre>
Reuse Instructions	- Describe how the program is used. Provide the declaration format, parameter values and types, and parameter limits. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.

Reuse Example	<pre> Instrucciones de reutilización */ /* void calcularParametros(String[] dataX, String[] dataY)*/ /* Propósito: calcular parámetros de regresión lineal */ /* Limitaciones: los arreglos deben tener la misma */ /* longitud y contener valores numéricos */ /* Retorna void, almacena resultados en variables de */ /* instancia */ /****** </pre>
Identifiers	Use descriptive names for all variables, function names, constants, and other identifiers. Avoid abbreviations or single letter variables.
Identifier Example	<pre> /** * Procesa los datos y calcula la regresión lineal y * correlación. * * @param dataX datos del eje X * @param dataY datos del eje Y * @param xk valor de xk para la predicción * @param labelX etiqueta para el eje X * @param labelY etiqueta para el eje Y */ private void procesarDatos(Data dataX, Data dataY, double xk, String labelX, String labelY) { EstimacionCorLineal estimacion = new EstimacionCorLineal(); String[] arrDataX = dataX.getAllData(); String[] arrDataY = dataY.getAllData(); estimacion.calcularParametros(arrDataX, arrDataY); mostrarResultados(estimacion, xk, labelX, labelY); } </pre>

(continued)

Coding Standard Template (continued)

Comments	• Document the code so that the reader can understand its operation. • Comments should explain both the purpose and behavior of the code. • Comment variable declarations to indicate their purpose.
Good Comment	<pre>/** * Muestra los resultados de los cálculos en consola. * * @param estimacion objeto con los cálculos realizados * @param xk valor de xk usado para la predicción * @param labelX etiqueta para el eje X * @param labelY etiqueta para el eje Y */</pre>
Bad Comment	
Major Sections	Precede major program sections by a block comment that describes the processing that is done in the next section
Example	<pre>/** * Clase principal que ejecuta los cálculos de regresión * lineal y correlación. * Implementa el Programa 3 del PSP (Personal Software * Process). * * @version 2.0.1 26/11/2025 * @author Bridget Mendez */ public class App { /** * Constructor por defecto. */ public App() { } /** * Punto de entrada principal del programa. * Muestra un menú para seleccionar qué tipo de cálculo * realizar. * * @param args argumentos de línea de comandos (no * utilizados) */ }</pre>
Blank Spaces	• Write programs with sufficient spacing so they do not appear crowded. • Separate every program construct with at least one space.

Indenting	• Indent every level of brace from the previous one. • Open and closing braces should be on lines by themselves and aligned with each other.
Indenting Example	<pre> /** * Verifica que un resultado esté dentro del margen de * error aceptable. * * @param calculado valor calculado * @param esperado valor esperado * @param nombre nombre del parámetro */ </pre>
Capitalization	• Capitalized all defines. • Lowercase all other identifiers and reserved words. • Messages being output to the user can be mixed-case so as to make a clean user presentation.
Capitalization Example	<pre> * Punto de entrada principal del programa. * Muestra un menú para seleccionar qué tipo de cálculo * realizar. * * @param args argumentos de línea de comandos (no * utilizados) */ </pre>